

Conversiono AI Platform

Epic 6: Security Hardening & GDPR

JWT Authentication, GDPR Compliance, Audit Logging

Jira Stories: CONV-56 through CONV-63

FastAPI | python-jose | passlib | bcrypt | slowapi

Sprint 6 - April 2026

10 Chapters | 4 Auth Endpoints | 3 GDPR Endpoints | 2 Middleware | 104 Tests

Table of Contents

Chapter 1	Overview
Chapter 2	JWT Authentication
Chapter 3	Auth API Endpoints
Chapter 4	GDPR Compliance
Chapter 5	Audit Logging
Chapter 6	Security Headers
Chapter 7	Rate Limiting
Chapter 8	Database Migration
Chapter 9	Configuration
Chapter 10	Testing & Verification

Chapter 1

Overview

1.1 What Epic 6 Delivers

Epic 6 transforms the Conversiono AI Platform from an open API into a production-grade, secure system ready for real merchant onboarding. It introduces a complete authentication layer using JSON Web Tokens (JWT), implements GDPR compliance endpoints mandated by European regulation, adds audit logging for accountability, hardens HTTP responses with security headers, and rate-limits sensitive endpoints to prevent abuse.

Before Epic 6, any HTTP client could access every endpoint. After Epic 6, merchants must register, authenticate, and present a valid JWT bearer token to access protected resources. Every state-changing request is logged for audit. Merchants can exercise their GDPR rights to export or delete personal data.

1.2 Why Security Matters for E-Commerce Analytics

An e-commerce analytics platform processes sensitive business data: conversion rates, revenue figures, customer session recordings, A/B test results, and intervention strategies. A breach exposes competitive intelligence and can violate customer privacy. For merchants operating in the EU, GDPR compliance is not optional - it carries fines of up to 4% of global annual turnover.

- Authentication prevents unauthorized access to merchant dashboards and APIs.
- Audit logging provides a forensic trail for incident response and compliance audits.
- Security headers mitigate common web attacks (XSS, clickjacking, MIME sniffing).
- Rate limiting protects against brute-force login attempts and API abuse.
- GDPR endpoints ensure legal compliance for EU merchants and their customers.

1.3 Jira Stories Covered

Story	Title	Status	Points
CONV-56	JWT Authentication Module	Done	8
CONV-57	Auth API Endpoints	Done	5
CONV-58	GDPR Compliance Endpoints	Done	8
CONV-59	Audit Logging Middleware	Done	5
CONV-60	Security Headers Middleware	Done	3
CONV-61	Rate Limiting Configuration	Done	5
CONV-62	Database Migration 004	Done	3
CONV-63	Integration Testing	Done	5

Key Achievement

All 8 stories completed with 104 tests passing and zero ruff lint errors.

Chapter 2

JWT Authentication

2.1 Architecture Overview

The authentication system is implemented in `backend/app/auth.py` and follows industry best practices for token-based authentication. It uses three core libraries: `passlib` for password hashing, `python-jose` for JWT encoding/decoding, and `bcrypt` as the underlying hashing algorithm.

The flow is straightforward: a merchant registers with email and password, the password is hashed with `bcrypt` before storage, and on login the server verifies the password and issues a signed JWT. Subsequent requests include this token in the Authorization header as a Bearer token.

2.2 Password Hashing with bcrypt

Passwords are never stored in plaintext. The `passlib` `CryptContext` is configured with `bcrypt` as the sole hashing scheme, and auto-deprecation is enabled so future algorithm upgrades are transparent.

```
from passlib.context import CryptContext

pwd_context = CryptContext(schemes=["bcrypt"], deprecated="auto")

def hash_password(password: str) -> str:
    """Hash a plaintext password using bcrypt."""
    return pwd_context.hash(password)

def verify_password(plain: str, hashed: str) -> bool:
    """Verify a plaintext password against a bcrypt hash."""
    return pwd_context.verify(plain, hashed)
```

`bcrypt` is a deliberately slow hashing function. Each hash computation takes ~100ms by default (tunable via work factor), making brute-force attacks computationally expensive. The auto-deprecation feature means that if a merchant logs in with a hash from an older scheme, `passlib` will transparently re-hash with the current scheme.

2.3 JWT Token Creation

Tokens are created using `python-jose` with the HS256 algorithm. Each token contains the merchant's email as the 'sub' (subject) claim and an expiration timestamp. The signing key is loaded from the `JWT_SECRET_KEY` configuration.

```
from jose import JWTError, jwt
from datetime import datetime, timedelta
from app.config import settings

def create_access_token(
    data: dict,
    expires_delta: timedelta | None = None
) -> str:
    to_encode = data.copy()
    expire = datetime.utcnow() + (
        expires_delta or timedelta(
            minutes=settings.JWT_ACCESS_TOKEN_EXPIRE_MINUTES
        )
    )
    to_encode.update({"exp": expire})
    return jwt.encode(
        to_encode,
        settings.JWT_SECRET_KEY,
        algorithm=settings.JWT_ALGORITHM
    )
```

2.4 Token Verification & Dependency Injection

FastAPI's dependency injection system is used to protect endpoints. The `get_current_merchant` dependency extracts the Bearer token from the Authorization header, decodes and validates it, then returns the merchant record from the database. If the token is missing, expired, or invalid, a 401 Unauthorized response is returned.

```
from fastapi import Depends, HTTPException, status
from fastapi.security import OAuth2PasswordBearer

oauth2_scheme = OAuth2PasswordBearer(tokenUrl="/auth/login")

async def get_current_merchant(
    token: str = Depends(oauth2_scheme),
    db: AsyncSession = Depends(get_db)
) -> Merchant:
    credentials_exception = HTTPException(
        status_code=status.HTTP_401_UNAUTHORIZED,
        detail="Could not validate credentials",
        headers={"WWW-Authenticate": "Bearer"},
    )
    try:
        payload = jwt.decode(
            token,
            settings.JWT_SECRET_KEY,
            algorithms=[settings.JWT_ALGORITHM]
        )
        email: str = payload.get("sub")
        if email is None:
            raise credentials_exception
    except JWTError:
        raise credentials_exception
    merchant = await get_merchant_by_email(db, email)
    if merchant is None:
        raise credentials_exception
    return merchant
```

Security Note

Always use HTTPS in production to prevent token interception.
Never log JWT tokens - they are bearer credentials.
Rotate JWT_SECRET_KEY periodically and on any suspected compromise.

Chapter 3

Auth API Endpoints

3.1 POST /auth/register

Creates a new merchant account. Accepts email, password, and company_name. Validates email format, checks for duplicates, hashes the password with bcrypt, and returns a JWT access token so the merchant is immediately logged in after registration.

Request Schema

```
POST /auth/register
Content-Type: application/json

{
  "email": "merchant@example.com",
  "password": "SecureP@ss123",
  "company_name": "Acme Store"
}
```

Response Schema (201 Created)

```
{
  "access_token": "eyJhbGciOiJIUzI1NiIs...\"",
  "token_type": "bearer",
  "merchant": {
    "id": "uuid-here",
    "email": "merchant@example.com",
    "company_name": "Acme Store",
    "onboarding_completed": false
  }
}
```

Error Responses

- 400 Bad Request - Invalid email format
- 409 Conflict - Email already registered
- 422 Unprocessable Entity - Missing required fields

3.2 POST /auth/login

Authenticates an existing merchant. Uses OAuth2 password flow compatible form data (username field maps to email). Returns a JWT access token on success. Failed attempts are rate-limited to prevent brute-force attacks.

Request Schema

```
POST /auth/login
Content-Type: application/x-www-form-urlencoded

username=merchant@example.com&password=SecureP@ss123
```

Response Schema (200 OK)

```
{
  "access_token": "eyJhbGciOiJIUzI1NiIs... ",
  "token_type": "bearer"
}
```

Error Responses

- 401 Unauthorized - Incorrect email or password
- 429 Too Many Requests - Rate limit exceeded

3.3 GET /auth/me

Returns the currently authenticated merchant's profile. Requires a valid JWT Bearer token in the Authorization header. This endpoint is the primary way the frontend dashboard fetches the merchant's identity after login.

Request

```
GET /auth/me
Authorization: Bearer eyJhbGciOiJIUzI1NiIs...
```

Response Schema (200 OK)

```
{
  "id": "uuid-here",
  "email": "merchant@example.com",
  "company_name": "Acme Store",
  "onboarding_completed": true,
  "gdpr_consent": true,
  "gdpr_consent_at": "2026-04-10T14:30:00Z"
}
```

3.4 POST /auth/onboarding-complete

Marks the merchant's onboarding as completed. Called after the merchant finishes the initial setup wizard in the dashboard. Sets the `onboarding_completed` flag to true in the database. Requires authentication.

Response Schema (200 OK)

```
{  
  "message": "Onboarding completed successfully",  
  "onboarding_completed": true  
}
```

Chapter 4

GDPR Compliance

4.1 Regulatory Background

The General Data Protection Regulation (GDPR) is a European Union regulation that governs how personal data must be handled. For Conversiono, this affects merchant account data, session analytics linked to end-users, and any personally identifiable information (PII) processed by the platform.

Three GDPR articles are directly implemented in Epic 6:

- Article 7 - Conditions for consent: explicit, informed, freely given consent
- Article 17 - Right to erasure ('right to be forgotten'): delete personal data on request
- Article 20 - Right to data portability: export personal data in a structured format

4.2 POST /auth/gdpr/consent

Records the merchant's explicit GDPR consent. This endpoint stores a boolean consent flag along with a timestamp indicating when consent was given. Consent can be granted or withdrawn by sending consent: true or consent: false.

```
POST /auth/gdpr/consent
Authorization: Bearer <token>
Content-Type: application/json

{
  "consent": true
}

# Response 200 OK
{
  "message": "GDPR consent recorded",
  "gdpr_consent": true,
  "gdpr_consent_at": "2026-04-10T14:30:00Z"
}
```

4.3 GET /auth/gdpr/export (Article 20)

Implements GDPR Article 20 - Right to Data Portability. Returns all personal data the platform holds about the merchant in a structured, commonly used, machine-readable JSON format. This allows the merchant to transfer their data to another service provider.

Data Included in Export

- Merchant profile: id, email, company_name, created_at, updated_at

- GDPR consent records: consent status and timestamps
- Onboarding status and completion date
- Sessions created by the merchant's SDK integration
- Experiments and their configurations
- Interventions triggered for the merchant's store
- Analytics aggregates and cohort definitions

```
GET /auth/gdpr/export
Authorization: Bearer <token>

# Response 200 OK
{
  "merchant": {
    "id": "uuid-here",
    "email": "merchant@example.com",
    "company_name": "Acme Store",
    "created_at": "2026-03-01T10:00:00Z"
  },
  "sessions_count": 1542,
  "experiments": [...],
  "interventions": [...],
  "exported_at": "2026-04-10T14:35:00Z"
}
```

4.4 DELETE /auth/gdpr/delete (Article 17)

Implements GDPR Article 17 - Right to Erasure. This is the most destructive endpoint in the system and permanently deletes all merchant data. The deletion is cascading and irreversible.

Deletion Order (Cascading)

1. All interventions associated with the merchant's experiments
2. All experiments owned by the merchant
3. All session events linked to the merchant's sessions
4. All sessions tracked for the merchant's store
5. All cohort analytics and aggregates
6. The merchant record itself (email, password_hash, company_name)

```
DELETE /auth/gdpr/delete
Authorization: Bearer <token>

# Response 200 OK
{
  "message": "All personal data has been deleted",
  "deleted_resources": {
    "merchant": true,
    "sessions": 1542,
    "experiments": 5,
    "interventions": 23
  }
}
```

Irreversible Operation

The GDPR delete endpoint permanently erases ALL merchant data.

There is no soft-delete or recovery mechanism.

The frontend should require explicit confirmation before calling this.

Chapter 5

Audit Logging

5.1 Why Audit Logging Matters

Audit logging provides an immutable record of all state-changing operations in the system. It is essential for security incident response, compliance audits (SOC 2, GDPR Article 30), debugging production issues, and understanding system usage patterns.

5.2 AuditLogMiddleware Implementation

The audit logging system is implemented as a Starlette/FastAPI middleware that intercepts every incoming HTTP request. It selectively logs requests that use state-changing HTTP methods: POST, PUT, PATCH, and DELETE. GET requests are excluded to avoid log volume explosion.

```
import logging
from starlette.middleware.base import (
    BaseHTTPMiddleware
)
from starlette.requests import Request

logger = logging.getLogger("audit")

class AuditLogMiddleware(BaseHTTPMiddleware):
    LOGGED_METHODS = {"POST", "PUT", "PATCH", "DELETE"}

    async def dispatch(self, request: Request, call_next):
        response = await call_next(request)
        if request.method in self.LOGGED_METHODS:
            logger.info(
                "AUDIT | %s %s | status=%d | ip=%s",
                request.method,
                request.url.path,
                response.status_code,
                request.client.host if request.client
                else "unknown",
            )
        return response
```

5.3 Log Format and Fields

Each audit log entry contains the following fields:

- Timestamp - ISO 8601 format from Python's logging configuration

- Log level - Always INFO for audit entries
- Prefix - 'AUDIT' marker for easy grep/filtering
- HTTP method - POST, PUT, PATCH, or DELETE
- URL path - The endpoint path that was accessed
- Status code - The HTTP response status code
- Client IP - The remote IP address of the requester

Example Log Output

```
2026-04-10 14:30:00 INFO audit AUDIT | POST /auth/register | status=201 | ip=192.168.1.1
2026-04-10 14:30:05 INFO audit AUDIT | POST /auth/login | status=200 | ip=192.168.1.1
2026-04-10 14:31:00 INFO audit AUDIT | DELETE /auth/gdpr/delete | status=200 | ip=192.168.1.1
```

Production Recommendation

In production, route audit logs to a separate log stream (e.g., CloudWatch, Datadog, or ELK stack) for long-term retention and search. Consider adding the authenticated merchant ID to each log entry for traceability.

Chapter 6

Security Headers

6.1 SecurityHeadersMiddleware

The SecurityHeadersMiddleware adds defensive HTTP headers to every response. These headers instruct browsers to enable built-in security mechanisms that mitigate common web vulnerabilities like cross-site scripting (XSS), clickjacking, and MIME type confusion attacks.

```
class SecurityHeadersMiddleware(BaseHTTPMiddleware):
    async def dispatch(self, request, call_next):
        response = await call_next(request)
        response.headers["X-Content-Type-Options"] = (
            "nosniff"
        )
        response.headers["X-Frame-Options"] = "DENY"
        response.headers["X-XSS-Protection"] = (
            "1; mode=block"
        )
        response.headers["Referrer-Policy"] = (
            "strict-origin-when-cross-origin"
        )
        response.headers["Cache-Control"] = (
            "no-store, no-cache, must-revalidate"
        )
        response.headers["Permissions-Policy"] = (
            "camera=(), microphone=(), geolocation=()"
        )
        return response
```

6.2 Header Explanations

X-Content-Type-Options: nosniff

Prevents browsers from MIME-sniffing a response away from the declared Content-Type. Without this header, a browser might interpret a JSON response as HTML, enabling XSS attacks through crafted API responses.

X-Frame-Options: DENY

Prevents the page from being embedded in an iframe. This is the primary defense against clickjacking attacks, where an attacker overlays a transparent iframe over a legitimate-looking page to trick users into clicking on hidden elements.

X-XSS-Protection: 1; mode=block

Enables the browser's built-in XSS filter and instructs it to block the page entirely rather than attempting to sanitize the content. While modern browsers are moving toward Content-Security-Policy, this header provides defense-in-depth for older browsers.

Referrer-Policy: strict-origin-when-cross-origin

Controls how much referrer information is included with requests. For same-origin requests, the full URL is sent. For cross-origin requests, only the origin (scheme + host + port) is sent. For downgrades (HTTPS to HTTP), no referrer is sent at all. This protects sensitive URL parameters from leaking to third-party services.

Cache-Control: no-store, no-cache, must-revalidate

Prevents caching of API responses. For a security-sensitive API that returns authentication tokens and personal data, caching could expose sensitive information through shared caches, browser history, or the back button. This is especially important for the /auth/me and /auth/gdpr/export endpoints.

Permissions-Policy: camera=(), microphone=(), geolocation=()

Restricts browser features that the application does not need. By explicitly denying camera, microphone, and geolocation access, the application reduces its attack surface. Even if an XSS vulnerability were exploited, the attacker could not access these sensitive device capabilities.

Chapter 7

Rate Limiting

7.1 Why Rate Limiting

Rate limiting is a critical security control that prevents abuse of the API. Without rate limits, an attacker could perform brute-force login attempts, overwhelm the server with expensive analytics queries, or exhaust database connections. Conversiono uses slowapi, which is built on top of the popular limits library and integrates natively with FastAPI.

7.2 slowapi Configuration

```
from slowapi import Limiter
from slowapi.util import get_remote_address

limiter = Limiter(
    key_func=get_remote_address,
    default_limits=["200/minute"],
    storage_uri="memory://",
)

# Attach to the FastAPI app
app.state.limiter = limiter
app.add_exception_handler(
    RateLimitExceeded, _rate_limit_handler
)
```

The limiter is configured with `get_remote_address` as the key function, which extracts the client IP from the request. In production behind a reverse proxy, this should be updated to use `X-Forwarded-For`. The default storage backend is in-memory, suitable for single-process deployments. For multi-worker deployments, switch to Redis storage.

7.3 Rate Tiers by Endpoint

Endpoint	Method	Limit	Reason
/auth/register	POST	5/minute	Prevent mass account creation
/auth/login	POST	10/minute	Prevent brute-force attacks
/auth/gdpr/export	GET	3/minute	Expensive data aggregation
/auth/gdpr/delete	DELETE	1/minute	Irreversible destructive op
/api/* (default)	ALL	200/minute	General API protection

7.4 Rate Limit Response

When a rate limit is exceeded, the API returns a 429 Too Many Requests response with a Retry-After header indicating when the client can retry.

```
HTTP/1.1 429 Too Many Requests
Retry-After: 60
Content-Type: application/json

{
  "error": "Rate limit exceeded",
  "detail": "10 per 1 minute",
  "retry_after": 60
}
```

Production Tip

For multi-worker deployments (gunicorn with uvicorn workers), switch `storage_uri` to `'redis://localhost:6379'` so rate limits are shared across all workers.

Chapter 8

Database Migration

8.1 Migration 004 Overview

Migration 004 extends the merchants table with columns required for authentication and GDPR compliance. It is designed as a non-destructive, additive migration - all new columns are nullable or have default values, so existing merchant records are not affected.

8.2 New Columns

Column	Type	Default	Purpose
password_hash	VARCHAR(255)	NULL	bcrypt-hashed password
gdpr_consent	BOOLEAN	FALSE	GDPR consent status
gdpr_consent_at	TIMESTAMP	NULL	When consent was given
onboarding_completed	BOOLEAN	FALSE	Onboarding wizard status

8.3 Migration Script

```
"""004 - Add auth and GDPR columns to merchants.

Revision ID: 004
Revises: 003
"""

from alembic import op
import sqlalchemy as sa

def upgrade():
    op.add_column("merchants", sa.Column(
        "password_hash", sa.String(255), nullable=True
    ))
    op.add_column("merchants", sa.Column(
        "gdpr_consent",
        sa.Boolean,
        server_default=sa.text("false"),
        nullable=False,
    ))
    op.add_column("merchants", sa.Column(
        "gdpr_consent_at", sa.DateTime, nullable=True
    ))
    op.add_column("merchants", sa.Column(
        "onboarding_completed",
        sa.Boolean,
        server_default=sa.text("false"),
        nullable=False,
    ))

def downgrade():
    op.drop_column("merchants", "onboarding_completed")
    op.drop_column("merchants", "gdpr_consent_at")
    op.drop_column("merchants", "gdpr_consent")
    op.drop_column("merchants", "password_hash")
```

8.4 Running the Migration

```
# Apply migration
alembic upgrade head

# Verify columns exist
alembic current

# Rollback if needed
alembic downgrade -1
```

Migration Safety

Always back up the database before running migrations in production.
The downgrade() function drops columns, which causes data loss.
Test migrations on a staging environment first.

Chapter 9

Configuration

9.1 JWT Settings

All JWT-related configuration is centralized in the Pydantic BaseSettings class in backend/app/config.py. Settings are loaded from environment variables or a .env file, with sensible defaults for development.

```
from pydantic_settings import BaseSettings

class Settings(BaseSettings):
    # JWT Configuration
    JWT_SECRET_KEY: str = "change-me-in-production"
    JWT_ALGORITHM: str = "HS256"
    JWT_ACCESS_TOKEN_EXPIRE_MINUTES: int = 30

    # Database
    DATABASE_URL: str = "sqlite+aiosqlite:///./conv.db"

    class Config:
        env_file = ".env"
        case_sensitive = True
```

9.2 Configuration Parameters Explained

JWT_SECRET_KEY

The secret key used to sign and verify JWT tokens. Must be a long, random string in production. If this key is compromised, an attacker can forge valid tokens for any merchant. Generate with: `openssl rand -hex 32`

JWT_ALGORITHM

The algorithm used for JWT signing. HS256 (HMAC-SHA256) is the default and is appropriate for single-service architectures. For microservice architectures where multiple services need to verify tokens, consider RS256 (RSA) which uses asymmetric key pairs.

JWT_ACCESS_TOKEN_EXPIRE_MINUTES

Token lifetime in minutes. The default is 30 minutes. Shorter lifetimes improve security (less time for a stolen token to be used) but hurt user experience (more frequent re-authentication). In production, consider implementing refresh tokens for a better balance.

9.3 Environment Variables (.env)

```
# .env file for production
JWT_SECRET_KEY=a1b2c3d4e5f6...long-random-hex-string
JWT_ALGORITHM=HS256
JWT_ACCESS_TOKEN_EXPIRE_MINUTES=30
DATABASE_URL=postgresql+asyncpg://user:pass@host/db
```

Security Warning

NEVER commit .env files to version control.

NEVER use the default JWT_SECRET_KEY in production.

Use a secrets manager (AWS Secrets Manager, Vault) for production keys.

Chapter 10

Testing & Verification

10.1 Test Suite Overview

Epic 6 maintains the comprehensive test suite established in previous epics, with all 104 tests passing. The tests cover unit tests for authentication functions, integration tests for API endpoints, middleware tests for security headers and audit logging, and end-to-end auth flow tests.

10.2 Testing the Auth Flow

Step 1: Register a New Merchant

```
import httpx

async with httpx.AsyncClient(base_url="http://localhost:8000") as client:
    # Register
    resp = await client.post("/auth/register", json={
        "email": "test@example.com",
        "password": "TestP@ss123",
        "company_name": "Test Store"
    })
    assert resp.status_code == 201
    token = resp.json()["access_token"]
```

Step 2: Access Protected Endpoints

```
# Use the token
headers = {"Authorization": f"Bearer {token}"}
me = await client.get("/auth/me", headers=headers)
assert me.status_code == 200
assert me.json()["email"] == "test@example.com"
```

Step 3: Test GDPR Endpoints

```
# Record consent
consent = await client.post(
    "/auth/gdpr/consent",
    json={"consent": True},
    headers=headers
)
assert consent.status_code == 200

# Export data
export = await client.get(
    "/auth/gdpr/export", headers=headers
)
assert export.status_code == 200
assert "merchant" in export.json()
```

Step 4: Verify Unauthorized Access is Blocked

```
# Without token - should fail
resp = await client.get("/auth/me")
assert resp.status_code == 401

# With invalid token - should fail
bad_headers = {"Authorization": "Bearer invalid.token.here"}
resp = await client.get("/auth/me", headers=bad_headers)
assert resp.status_code == 401
```

10.3 Running the Full Test Suite

```
# Run all tests with verbose output
cd backend
python -m pytest tests/ -v --tb=short

# Expected output:
# 104 passed in 8.23s

# Run only auth tests
python -m pytest tests/ -v -k "auth"

# Run with coverage
python -m pytest tests/ --cov=app --cov-report=term-missing
```

10.4 Lint Verification

All code passes ruff linting with zero errors. Ruff is configured as the project's primary linter and is enforced in CI.

```
# Run ruff lint check
ruff check backend/

# Expected output:
# All checks passed!
```

10.5 Test Categories

Category	Count	Coverage Area
Unit - Password Hashing	6	hash_password, verify_password, edge cases
Unit - JWT Functions	8	create_token, decode_token, expiry, invalid tokens
Integration - Register	10	Success, duplicate email, invalid input, response schema
Integration - Login	8	Success, wrong password, nonexistent user, rate limit
Integration - GDPR	12	Consent, export, delete, unauthorized access
Integration - Middleware	10	Security headers, audit log entries, rate limits
End-to-End - Auth Flow	8	Register -> login -> access -> GDPR -> delete
Existing Tests	42	Sessions, experiments, interventions, analytics

CI/CD Integration

The test suite runs automatically in CI on every push.

Both pytest (104 tests) and ruff (0 errors) must pass for a merge.